人工智能原理与技术大作业实验报告

莫亦非
PB23151772
moyifei135@mail.ustc.edu.cn

金义博
PB23000086
jin218@mail.ustc.edu.cn

黄思睿
PB23151782
halo@mail.ustc.edu.cn

Abstract

本报告聚焦于人工智能领域中变分自编码器（VAE）的原理与应用。首先介绍了 KL 散度这一用于衡量分布相似度的概念，并用其来推导 VAE 的优化目标，同时结合 EM 算法，阐述了 VAE 通过优化证据下界（ELBO）来实现模型训练的数学原理。在图像生成任务方面，本报告探讨了 VAE 的架构设计，包括 Encoder 和 Decoder 的作用，以及损失函数的组成，同时提到了重参数化技巧来解决训练中的梯度传播问题。对于条件生成任务，报告对比了基于 MLP 和 CNN 的两种架构，MLP 通过独热编码处理标签并与图像特征拼接，而 CNN 则能更好地保留图像的空间结构信息，实验中通过调整参数确定了潜在维度、学习率等最优参数，结果显示 CNN 架构效果更优。最后，报告还提出了引入多尺度特征融合、注意力机制等未来改进方向。

***K*eywords** 图像生成 · VAE · MLP · CNN

1 预备知识与部分证明

1.1 VAE 的优化目标：KL 散度 (20pt)

我们首先需要的是，如何去衡量我们的分布与目标分布的相似度呢？这里就引入了一个概念：KL 散度

$$D_{KL}(p||q) = \int_x p(x) \log \frac{p(x)}{q(x)} dx = E_{x\sim p(x)}[\log \frac{p(x)}{q(x)}]$$

首先，证明一些 KL 散度的基本性质

1. 任意两个分布 p, q，$D_{KL}(p||q) \geq 0$，取 0 时当且仅当 $p = q$ (10pt)
证:

$$\begin{aligned} D_{KL}(p||q) &= \int_x p(x) \log \frac{p(x)}{q(x)} dx \\ &= -\int_x p(x) \log \frac{q(x)}{p(x)} dx \\ &\geq \int_x p(x)(1 - \frac{q(x)}{p(x)}) dx \\ &= \int_x p(x) - q(x) dx \\ &= \int_x p(x) dx - \int_x q(x) dx \end{aligned}$$

由密度函数的性质可知 $\int_x p(x)dx = 1$, 故

$$D_{KL}(p||q) \geq \int_x p(x)dx - \int_x q(x)dx = 1 - 1 = 0$$

可以发现，取等条件为 $\log \frac{q(x)}{p(x)} = \frac{q(x)}{p(x)} - 1$，当且仅当 $\frac{q(x)}{p(x)} = 1$，即 $p(x) = q(x)$ 时成立。

2. 对两个一维正态分布(高斯分布), $p(x) \sim \mathcal{N}(\mu_1, \sigma_1^2), q(x) \sim \mathcal{N}(\mu_2, \sigma_2^2)$,其 KL 散度 $D_{KL}(p\|q) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$。(10pt)

$$\log \frac{p(x)}{q(x)} = \frac{(x - \mu_2)^2}{2\sigma_2^2} - \frac{(x - \mu_1)^2}{2\sigma_1^2} + \log \frac{\sigma_2}{\sigma_1}$$

下面对三项分别计算期望:
对于第一项:

$$\mathbb{E}_{x \sim p(x)} \left[\frac{(x - \mu_2)^2}{2\sigma_2^2} \right] = \frac{\mathbb{E}_{x \sim p(x)} \left[(x - \mu_2)^2 \right]}{2\sigma_2^2}$$

令 $(x - \mu_2)^2 = (x - \mu_1 + \mu_1 - \mu_2)^2$ 并展开后取期望:

$$\mathbb{E}_{x \sim p(x)} \left[(x - \mu_2)^2 \right] = \mathbb{E}_{x \sim p(x)} \left[(x - \mu_1)^2 \right] + 2(\mu_1 - \mu_2) \mathbb{E}_{x \sim p(x)} \left[x - \mu_1 \right] + (\mu_1 - \mu_2)^2$$

由于 $\mathbb{E}_{x \sim p(x)} \left[x - \mu_1 \right] = 0$，所以:

$$\mathbb{E}_{x \sim p(x)} \left[(x - \mu_2)^2 \right] = \sigma_1^2 + (\mu_1 - \mu_2)^2$$

因此:

$$\frac{\mathbb{E}_{x \sim p(x)} \left[(x - \mu_2)^2 \right]}{2\sigma_2^2} = \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2}$$

对于第二项:

$$\mathbb{E}_{x \sim p(x)} \left[-\frac{(x - \mu_1)^2}{2\sigma_1^2} \right] = -\frac{\mathbb{E}_{x \sim p(x)} \left[(x - \mu_1)^2 \right]}{2\sigma_1^2}$$

由于 $\mathbb{E}_{x \sim p(x)} \left[(x - \mu_1)^2 \right] = \sigma_1^2$（方差定义），所以:

$$\mathbb{E}_{x \sim p(x)} \left[-\frac{(x - \mu_1)^2}{2\sigma_1^2} \right] = -\frac{\sigma_1^2}{2\sigma_1^2} = -\frac{1}{2}$$

对于常数项

$$\mathbb{E}_{x\sim p(x)}\left[\log\frac{\sigma_2}{\sigma_1}\right]=\log\frac{\sigma_2}{\sigma_1}$$

最终

$$\boxed{D_{KL}(p\|q)=\log\frac{\sigma_2}{\sigma_1}+\frac{\sigma_1^2+(\mu_1-\mu_2)^2}{2\sigma_2^2}-\frac{1}{2}}$$

3. 这里直接给出高维的结论，假设 $p(\mathbf{x})\sim\mathcal{N}(\mu_1,\Sigma_1),1(\mathbf{x})\sim\mathcal{N}(\mu_2,\Sigma_2)$，这里的 μ 为 n 维向量，Σ 为 n 维方阵，则

$$D_{KL}(p||q)=\frac{1}{2}\log\frac{|\Sigma_2|}{|\Sigma_1|}+\frac{1}{2}tr(\Sigma_2^{-1}\Sigma_1)+(\mu_1-\mu_2)^T\Sigma_2^{-1}(\mu_1-\mu_2)-\frac{1}{2}n$$

1.2 EM 算法数学原理 (10pt)

EM 算法本质上其实也是一种 MLE（极大似然估计），不过有些许不同。我们希望从采样的样本数据 x 来估计这个分布的参数 θ，也就是最大化 $p(\theta|x)$，极大似然估计的想法就是

$$\theta^*=\arg\max_\theta p(\theta|x)=\arg\max_\theta\frac{p(\theta)p(x|\theta)}{p(x)}=\arg\max_\theta p(x|\theta)=\arg\max_\theta\log p(x|\theta)$$

但现在在 EM 算法中有些许不同，我们没办法直接得到 $p(x|\theta)$，因为 x 不是能够直接建模的变量，而是需要从一个可建模可观测或者说可控制的隐变量 z（服从已知分布 $q(z)=p(z|\theta)$）过渡得到。所以我们现在要优化的目标可以进一步展开

$$\begin{aligned}
L(\theta)&=\log p(x|\theta)\\
&=\int_z q(z)\log p(x|\theta)dz\\
&=\int_z q(z)\log\frac{p(x,z|\theta)}{p(z|x,\theta)}dz\\
&=\int_z q(z)\log\frac{q(z)\cdot p(x|z,\theta)}{p(z|x,\theta)}dz\\
&=\int_z q(z)\left(\log p(x|z,\theta)+\log\frac{q(z)}{p(z|x,\theta)}\right)dz\\
&=ELBO+\int_z q(z)\log\frac{q(z)}{p(z|x,\theta)}dz
\end{aligned}$$

前面一项 $ELBO=\int_z q(z)\log p(x|z,\theta)dz$;
后面一项是一个 $D_{KL}(q(z)||p(z|x,\theta))$，这一项会在当 $q(z)$ 和 $p(z|x,\theta)$ 相同时取 0。所以 EM 算法的两步骤其实是在做这样的事：EM 算法中的隐藏分布 $q(z)$ 是一个完全可控的分布，所以我们希望能够同时优化 q,θ，让 ELBO 直接作为优化目标，写成数学形式就是

$$max_{q,\theta}ELBO$$

1. E-step：我们直接令 $q^{(t+1)}(z)=p(z|x,\theta^{(t)})$，这里的 $\theta^{(t)}$ 表示当前的参数（需要不断更新），这样我们的后一项直接为 0 了

2. M-step：我们优化这个 ELBO 即可 $\theta^{(t+1)}=\arg\max_\theta J(\theta,q^{(t+1)})=\arg\max_\theta ELBO$

1.3 图像生成任务的基本原理

基本假设：我们认为，图像在数学层面上 (C(channel，RGB 是 3，灰度图会是 1) × W (高度)× H (宽度)) 是服从一个高维分布的。我们用随机变量 $\mathbf{x}$ 来表示。而对于我们给定的图像数据集里 (这个数据集可大可小)，我们认为我们的分布为 $p_{data}(\mathbf{x})$。我们的任务就是建模这个分布，用我们带有可训练的参数的 $p_\theta(\mathbf{x})$ 去尽可能逼近它。

1.4 VAE 架构与 Motivation

我们希望从一个我们可控的简单分布 z（一般可以是一个简单的标准高斯分布），过渡到我们需要逼近的 $p_{data}(x)$，而建模这个过程的 $p_\theta(x|z)$，就是我们的模型要做的。

1.5 损失函数的推导 (10pt)

根据上述分析，我们的优化目标可以写成：

$$\theta^* = \arg\min_\theta D_{KL}(p_{data}||p_\theta)$$

根据 DL 散度的定义：

$$\begin{aligned} D_{KL}(p_{data}||p_\theta) &= \int_x p_{data}(x)\log\frac{p_{data}(x)}{p_\theta(x)}dx \\ &= \int_x p_{data}(x)\log p_{data}(x)dx - \int_x p_{data}(x)\log p_\theta(x)dx \\ &= \mathbb{E}_{x\sim p_{data}}[\log p_{data}(x)] - \mathbb{E}_{x\sim p_{data}}[\log p_\theta(x)] \end{aligned}$$

其中，前一项与 θ 无关，故优化目标相当于

$$\boxed{\theta^* = \arg\max_\theta \mathbb{E}_{x\sim p_{data}}[\log p_\theta(\mathbf{x})]}$$

这个式子对于实际的操作就方便太多了，这里的期望我们可以通过采样来近似，我们从 p_{data} 中直接采样即可，然后通过对数似然的方法优化，也就是从数据集中随机采样 $\{\mathbf{x}^1, \mathbf{x}^2, \mathbf{x}^3, \cdots, \mathbf{x}^m\}$，来最大化 $\sum_{i=1}^m \log p_\theta(\mathbf{x}^i|\theta)$

但是还有个问题，我们是无法从 Input 的 $\mathbf{x}$ 直接得到最后的 Output，我们在使用模型做生成的时候，只能从中间的 $\mathbf{z}$，我们假设服从分布 $q(\mathbf{z})$ (实际多数为标准高斯分布)，来生成。也就是说，我们无法直接使用极大似然估计的方法优化 p_θ，为了考虑到中间的 $\mathbf{z}$，我们就需要模仿 EM 算法的优化方法

$\mathcal{L}(\theta) = \log p_\theta(\mathbf{x}) = \int_{\mathbf{z}} q(\mathbf{z})\log p_\theta(\mathbf{x}|\theta)d\mathbf{z} = ELBO + D_{KL}(q(\mathbf{z})||p(\mathbf{z}|\mathbf{x},\theta))$

一方面与 EM 算法相同，我们只想要去优化 ELBO。但另一方面，我们很难用当前的 $p(x,\theta)$（这个我们也写不出来）去估计 $q(z)$。某种程度上，这一项是无法通过人为来计算修改的，所以就直接不管了，相当于我们去优化一个下界

那么我们就给 ELBO 拆开，就可以写成我们需要的优化目标了

$$\min_{\theta,\phi} -ELBO = \mathcal{L} = \underbrace{\mathbb{E}_{\mathbf{z}\sim q_\phi(\mathbf{z}|\mathbf{x})}[-\log p_\theta(\mathbf{x}|\mathbf{z})]}_{reconstruction} + \underbrace{D_{KL}(q_\phi(\mathbf{z}|\mathbf{x})||p(\mathbf{z}))}_{regularization}$$

我们用两个网络去建模我们需要的分布，一个 $p_\theta(\mathbf{x}|\mathbf{z})$，叫做 Decoder，一个 $q_\phi(\mathbf{z}|\mathbf{x})$，叫做 Encoder。

1.6 图像生成任务的具体使用

推导了 VAE 的基本原理以及损失函数后，我们再来看看在图像生成任务中如何使用这个模型。

我们将所有的分布都用高斯分布来建模，其中我们的损失函数中一共涉及三个分布：1. 隐变量/压缩后的信息 $p(\mathbf{z})$, 2.Decoder$p_\theta(\mathbf{x}|\mathbf{z})$, 3.Encoder$q_\phi(\mathbf{z}|\mathbf{x})$

1. $p(\mathbf{z})$ 我们一般直接建模成一个标准高斯分布 $\mathcal{N}(\mathbf{0}, I)$
2. $p_\theta(\mathbf{x}|\mathbf{z})$ 我们假设成 $\mathcal{N}(\mu_\theta(\mathbf{z}), \sigma^2 I)$，其中 μ 是神经网络的输出，σ 是一个人为设定的超参数

   > 至于为什么 σ 是超参数而不是神经网络输出，一方面在于最后的损失函数中，σ 只是正则化项的一个系数，另一方面在于我们实际在使用模型进行生成的时候，重点在于这个分布的期望，可以理解为模型所认为**最理想**的生成图像，也是我们要的。当然，我们可以在其上加上任意的方差

3. $q_\phi(\mathbf{z}|\mathbf{x})$ 我们假设成 $\mathcal{N}(\mu_\phi(\mathbf{x}), \sigma_\phi(\mathbf{x})I)$，我们神经网络的输出就是 μ, σ 这里我们假设协方差矩阵为单位阵形式，可以很大程度上减少模型输出，有效防止过拟合

最后我们的损失函数可以化简为

$\mathcal{L}(\theta, \phi, \mathbf{x}) = \sum_{i=1}^{d} \frac{1}{2}(-1 + (\sigma_\phi^{(i)})^2 + (\mu_\phi^{(i)})^2 - \log(\sigma_\phi^{(i)})^2) + \frac{1}{2\sigma^2}||\mathbf{x} - \mu_\theta(\mathbf{z})||_2^2$

注：这里需要一个叫做重参数化技巧 (reparameterization trick)，因为这里的 $\mathbf{z}$ 是隐藏 code 的一个采样，所以不能直接使用采样的方式来进行使用，否则梯度无法反向传播回去，但我们知道 $\mathbf{z}|\mathbf{x}$ 的分布，所以完全可以合成一下

$\mathbf{z} = \mu_\theta(\mathbf{x}) + \epsilon \cdot \sigma_\theta(\mathbf{x}), \quad \epsilon \sim \mathcal{N}(\mathbf{0}, I)$，这样我们只需要采样 ϵ 就可以了

2 条件生成任务的方法论设想

2.1 对标签进行独热编码的 MLP-CVAE 条件生成任务

2.1.1 使用 One-Hot 编码对标签进行转换

- 输入标签：整数形式 (0-9)
- 转换结果：10 维向量
- 示例：

```
标签 5 -> [0,0,0,0,0,1,0,0,0,0]
标签 3 -> [0,0,0,1,0,0,0,0,0,0]
```

使用 One-Hot 编码具有以下优势;

1. **类别等距性**：
   每个类别之间的欧氏距离相等，避免了数字标签带来的大小关系影响。
2. **独立维度表示**：
   采用类别 - 独立维度的设计，可使各特征维度间的干扰被规避，为网络学习类别特定特征构建了更清晰的空间，助力模型精准捕捉类别专属模式。
3. **梯度传播独立性**：
   每个类别有独立的梯度路径，便于网络学习类别特征，提高训练稳定性。

2.1.2 直接拼接图像和标签特征

编码器通过拼接图像和标签特征将图像和标签映射到潜在空间分布参数，解码器从潜在向量和标签重建图像。

- 编码结构设计

```python
def encode(self, x, labels):
    if labels.dim() == 2 and labels.size(1) == 1:
        labels = labels.squeeze(1)  # 将 [batch_size, 1] 变为 [batch_size]
    labels_onehot = F.one_hot(labels, num_classes=self.num_classes).float()
    x_with_labels = torch.cat([x, labels_onehot], dim=1)
    h = self.act(self.fc1(x_with_labels))
    mu = self.fc2_mu(h)
    log_var = self.fc2_logvar(h)
    return mu, log_var
```

- 解码结构设计

```python
def decode(self, z, labels):
    if torch.is_tensor(labels) and labels.dim() == 0:
        labels = labels.unsqueeze(0)  # 标量 -> [1]
    if labels.dim() == 2:
        if labels.size(1) == 1:
            labels = labels.squeeze(1)  # [batch_size, 1] -> [batch_size]
        else:
            raise ValueError(f"二维标签形状错误：应为[batch_size, 1]，实际形状 {labels.shape}")
    elif labels.dim() != 1:
        raise ValueError(f"标签维度错误：应为标量、一维或二维([batch_size, 1])，实际维度 {labels.dim()}")
    labels_onehot = F.one_hot(labels, num_classes=self.num_classes).float()
    z_with_labels = torch.cat([z, labels_onehot], dim=1)
    h = self.act(self.fc3(z_with_labels))
    recon_x =  torch.sigmoid(self.fc4(h))  # Using sigmoid to constrain the output to [0,1]
    return recon_x.view(-1, 1, 28, 28)
```

这么做的好处有：

1. **实现难度低，上手快**：
   这种方法不需要搭建复杂的特征融合结构，比如不需要额外设计注意力机制或门控网络，直接用张量拼接就能完成操作。因为少了很多复杂的模块，计算量小，运行速度快，在调试和优化模型时也更方便。我们在实验过程中发现，相比于其他融合方法，直接拼接能让代码更简洁，遇到问题时也更容易定位和解决。

2. **信息保留完整**：
   直接拼接不会对原始的图像和标签特征进行删减或压缩，能保留全部的原始信息。这样一来，模型就有机会充分学习到图像细节和标签语义之间的所有关联，不用担心因为特征处理导致重要信息丢失。在我们的实验里，用这种方式训练的模型，对细节的捕捉能力明显更好。

3. **运行效率更高**：
   直接拼接只需要一步操作，不需要像其他方法那样，反复对特征进行变换和映射。因为少了中间步骤，不仅计算速度变快，运行过程中占用的内存也更少。在处理大规模数据时，这个优势就更明显了，我们的模型训练时间缩短了近 20% ，电脑也不容易因为内存不足卡顿。

2.2 从 MLP 到 CNN：我们的架构升级之路

我们一开始对条件生成任务的实现是基于 MLP 架构的，但是在完成了基础的 MLP 条件 VAE 后，我们发现得分并不理想，一个很符合直觉的问题是：MLP 把 28×28 的图像直接拉平成 784 维的向量，这样做虽然简单，但是完全丢失了图像的空间结构信息。想象一下，原本相邻的像素点突然变成了向量中相距很远的元素，这对于图像处理来说显然是不合理的。

于是我们决定引入 CNN 架构，让模型能够更好地理解图像的空间关系。

2.2.1 CNN encoder

我们设计的 CNN 编码器包含三个卷积层，每一层都有自己的” 职责”：

```python
# 第一层：提取基础特征
self.enc_conv1 = nn.Conv2d(self.channels + 1, 32, kernel_size=3, stride=2, padding=1)
# 第二层：学习更复杂的形状
self.enc_conv2 = nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1)
# 第三层：捕捉高级语义特征
self.enc_conv3 = nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1)
```

整个特征提取过程就像这样：

- **输入图像**：28×28 的手写数字
- **第一层卷积**：变成 $14 \times 14 \times 32$，主要识别大体的基本元素
- **第二层卷积**：变成 $7 \times 7 \times 64$，开始识别局部形状
- **第三层卷积**：还是 7×7 但通道数增加到 128，学习完整数字的特征模式

这样设计的好处是显而易见的：

1. **保持了图像的空间结构**：
   不像 MLP 那样粗暴地拉平，CNN 通过卷积核在图像上滑动，始终保持着像素之间的邻近关系。这样模型就能学会" 左边有一条竖线，右边有一个圆圈，组合起来可能是数字 9" 这样的空间逻辑。

2. **参数效率大幅提升**：
   这点让我们印象特别深刻。同样的表达能力，我们的 CNN 版本只需要约 20 万个参数，而 MLP 版本需要 65 万个参数。虽然 CNN 计算量更大，训练明显更慢，但参数少，占用内存更少，也不容易过拟合。

3. **学习过程更符合直觉**：
   人类识别数字的时候，也是先看局部特征（比如直线、弯曲），再组合成完整的数字。CNN 的层次化特征提取正好模拟了这个过程。

2.2.2 标签信息的融入

我们将标签融入的方法是采用双通道拼接，即将原始图像和标签信息在通道维度上拼接，形成一个具有两个通道的输入张量。这种设计允许卷积神经网络同时处理视觉数据和额外的条件信息。

```
self.encoder = nn.Sequential(
    nn.Conv2d(self.channels + 1, 32, kernel_size=3, stride=2, padding=1),  # 2个通道：图像+标签
    nn.LeakyReLU(0.2),
    nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),
    nn.BatchNorm2d(64),
    nn.LeakyReLU(0.2),
    nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),
    nn.LeakyReLU(0.2),
    nn.AdaptiveAvgPool2d((3, 3))  # 强制输出3x3尺寸
)
```

我们选择在一开始就加入标签信息而不是在 CNN 特征提取完成后再融合标签，主要的考虑是：

- 标签信息在特征提取初期即参与卷积运算，卷积核可直接学习标签与图像像素的空间对应关系。
- 设生成器损失为 $L(G)$，双通道输入下 $\frac{\partial L}{\partial label}$ 可直接通过第一层卷积传播，而编码器后融合需经过多层非线性变换，梯度易消失。
- 代码简洁清晰。

2.2.3 decoder 实现

```
self.decoder_fc = nn.Linear(latent_dim + num_classes, feature_dim)  # 输入增加标签维度
self.decoder = nn.Sequential(
```

```
    nn.ConvTranspose2d(128, 64, kernel_size=3, stride=2, padding=1, output_padding=1),
    nn.BatchNorm2d(64),
    nn.ReLU(),
    nn.ConvTranspose2d(64, 32, kernel_size=3, stride=2, padding=1, output_padding=1),
    nn.BatchNorm2d(32),
    nn.ReLU(),
    nn.ConvTranspose2d(32, self.channels, kernel_size=3, stride=2, padding=1),
    nn.AdaptiveAvgPool2d((img_size, img_size)),
    nn.Sigmoid()
)
```

Decoder 的设计完全是 Encoder 的" 逆过程"，需要从潜在向量和标签生成具体的图像，从低维度慢慢扩展到可以重塑成特征图的维度：$7 \times 7 \to 14 \times 14 \to 28 \times 28$

2.2.4 遇到的问题与解决方案

在实际编程过程中，我们遇到了一些问题，也找到了对应的解决方案：

1. **张量维度不匹配问题**：在利用 encoder 进行训练时传入的 label 是带 batch size 维度的张量，而 inference.py 中的 generate_images() 函数中传入的 label 为标量，故在 decoder 中需要对 label 进行维度匹配操作：

```
if torch.is_tensor(labels) and labels.dim() == 0:
    labels = labels.unsqueeze(0)  # 标量 -> [1]
    # 2. 处理一维/二维张量标签
if labels.dim() == 2:
    if labels.size(1) == 1:
        labels = labels.squeeze(1)  # [batch_size, 1] -> [batch_size]
    else:
        raise ValueError(f"二维标签形状错误：应为[batch_size, 1]，实际形状 {
            labels.shape}")
elif labels.dim() != 1:
    raise ValueError(f"标签维度错误：应为标量、一维或二维([batch_size, 1])，实际
        维度 {labels.dim()}")
```

2. **显存不足报错**由于进行实验的计算机显卡显存只有 4G，故在默认 batch size 为 2048 时会出现显存不足报错，需要减小默认 batch size。

3. **激活函数的选择**：我们选择了 LeakyReLU(0.2) 而不是普通的 ReLU，因为 LeakyReLU 允许负值有小幅的梯度，能有效避免" 死神经元" 问题。输出层用 Sigmoid 确保像素值在 [0,1] 范围内。

4. **编写脚本进行调参**为了方便调参我们编写了如下所示的脚本进行调参：

```
#Genwithlabel.sh
python train.py --task Genwithlabel --epochs 20 --latent_dim 16 --batch_size
    1024 --lr 3e-3 --var 0.2
sleep 1
```

```
python inference.py --task Genwithlabel --latent_dim 16 --checkpoint_path
    Genwithlabel/test/epoch_100.pth --save_dir Genwithlabel/test
```

3 实验调参

3.1 模型选择

在使用默认参数的情况下，在两种任务中 CNN 方法的表现都比 MLP 方法获取了更高的分数，故最终两种任务我们都选择 CNN 方法实现。

3.2 调参思路

首先我们先对 epoch 进行调参，发现随着训练轮次的增加，模型得分也一直增加。由此我们先固定一个大小适中的轮次大小（20），接着对其他参数进行调参，最后在使用一个较大的轮次作为最终轮次参数。

其他参数的调参原则是对默认参数的正负一个数量级进行指数采样，若能找到极大值则采用极大值，若模型得分随着该参数大小递增或递减则先取一个适中值对其他参数进行调参，最后再采取较大或较小值。

3.2.1 latent dim

latent dim 即为潜在维度大小，即通过算法将原始高维数据压缩或映射到低维空间后所保留的特征维度数量。由于最终得分计算中鼓励减小 latent dim 大小，故在其分布范围内模型得分一定会出现极大值。

表 1: latent dim 与得分对应表

latent dim	learning rate	batch size	epoch	var	模型得分
8	3e-3	1024	20	0.2	24.56
16	3e-3	1024	20	0.2	26.99
32	3e-3	1024	20	0.2	25.50
64	3e-3	1024	20	0.2	24.82

由上表得最适合的 latent dim 大小应为 16。

3.2.2 learning rate

由于 learning rate 变化较为敏感，故采用线性采样方法。

由上表得最合适的 learning rate 大小应为 3e-3。

表 2: learning rate 与得分对应表

latent dim	learning rate	batch size	epoch	var	模型得分
16	1e-3	1024	20	0.2	24.99
16	2e-3	1024	20	0.2	25.82
16	3e-3	1024	20	0.2	26.99
16	4e-3	1024	20	0.2	26.64
16	5e-3	1024	20	0.2	26.14

3.2.3 batch size

采用较小的 batch size 采样噪声较大，导致对 KL 散度的估计不稳定，可能使模型过度关注重建损失(后验坍塌)。而采用较大的 batch size 采样更接近真实分布，KL 散度估计更准确，但可能陷入局部最优。由上表得最合适的 batch size 大小应为 256。

表 3: batch size 与得分对应表

latent dim	learning rate	batch size	epoch	var	模型得分
16	3e-3	128	20	0.2	27.48
16	3e-3	256	20	0.2	27.65
16	3e-3	512	20	0.2	27.56
16	3e-3	1024	20	0.2	26.99
16	3e-3	2048	20	0.2	25.17

3.2.4 var

var 较小时，潜在分布集中，生成样本接近训练数据的平均特征，多样性低。var 较大时，潜在分布分散，采样点更随机，生成样本多样性高，但可能偏离真实数据分布。

表 4: var 与得分对应表

latent dim	learning rate	batch size	epoch	var	模型得分
16	3e-3	256	20	0.1	25.44
16	3e-3	256	20	0.2	27.65
16	3e-3	256	20	0.3	27.25
16	3e-3	256	20	0.4	26.32

由上表得最合适的 var 大小应为 0.2。

4 总结

4.1 CNN 和 MLP 两种架构的对比

为了实现了这两种架构，我们学习了很多，也了解了很多他们各自的优缺点。

从实际体验来看，CNN 条件 VAE 虽然实现起来最复杂，但效果确实是最好的。特别是在生成图像的清晰度和条件控制的精确性上，有质的提升。当然，如果只是快速验证想法，MLP 版本还是很有用的。

4.2 可改进方向

1. **多尺度特征融合：**引入 U-Net 式跳连接（Skip Connection），保留底层细节特征。
2. **注意力机制：**添加 Self-Attention 或 Squeeze-and-Excitation（SE）模块，增强对细节的建模能力。
3. **量化潜在空间：**借鉴 VQ-VAE，使用离散码本替代连续分布，提升图像清晰度。

5 课程反馈

完成本次大作业花费时长总共为 50 小时（相当酣畅淋漓的一次 coding 了）。从小组成员刚拿到题目对着 VAE 三个字母一头雾水大眼瞪小眼，到慢慢磨洋工把代码架构看懂，期间翻阅了数篇论文和博客终于理解（感觉实验指导的内容还是很有限，需要自己花很多功夫去理解这个任务，anyway 还是很有收获）。

这学期在数据分析与实践这门课里学习到了 CNN 方法，这一次在图像生成任务中真正运用才体会到教科书中短短几句“适用于图像任务”具体是如何体现，有一种真的用上了所学知识的“幸福感”。

这学期选择 AI 这门课感觉对于自己来说还是不小的挑战，但是磕磕碰碰修修改改也算是有惊无险（没有退课）地走到了最后，很有一些感动吧。从第一次的地图规划看着项目欲哭无泪到第三次看见自驾小车跑起来的时候的欣喜，发觉能力也就被这么润物细无声的培养起来了，至少知道大段天书代码也并没有非常可怕。虽然很难说自己的水平提升了多少，但是至少学习的勇气有被锻炼到。

一个又一个实验下来，助教老师们都很用心，指导文档有不断进步，给到的 Q&A 也越来越详尽。同时也感谢在此过程中给到我们很多帮助的头部大佬同学们，真是泪中有笑的一次课程体验啊！